Periodic Net Symmetry Group Specification

Stage 7R Revision: Core Finite Group Without Ring-Derived Payloads

mdstats

2026-07-18

Contents

Purpose and boundary

`net_symmetry.py` represents the finite group of normalized periodic-net automorphisms modulo common lattice translations. The result belongs to one immutable `PeriodicNetView` and contains only group-theoretic data intrinsic to that view.

Runtime/API target:

`mdstats 0.19.19a0`

Primitive-ring actions are no longer fields of `PeriodicNetSymmetry`. They are a separate catalog-bound derived index specified by `primitive_ring_symmetry_spec.md`.

Mathematical object

A lifted periodic automorphism acts as

$$g(i, \mathbf{n}) = \left(\pi_g(i), A_g\mathbf{n} + \boldsymbol{\tau}_i^g\right), \qquad A_g \in GL(3, \mathbb{Z}).$$

The full infinite automorphism group contains every lattice translation. The module stores finite representatives of

$$\mathrm{Aut}(\tilde{G}, \sigma_V, \sigma_E)/\Lambda$$

for the subgroup generated by validated input operations.

Deterministic translation gauge

Choose one deterministic anchor vertex i_0 and normalize every representative by

$$\boldsymbol{\tau}_{i_0} = \mathbf{0}.$$

Operations differing only by a common output translation therefore have one canonical record.

Composition and cocycle

For `outer = g` and `inner = h`,

$$A_{g\circ h} = A_g A_h,$$

$$\tau_i^{g\circ h} = A_g \tau_i^h + \tau^g_{\pi_h(i)}.$$

Normalization may remove a common translation. Hence normalized representatives satisfy

$$\hat{g}\,\hat{h} = T_{\mathbf{c}(g,h)}\widehat{gh}.$$

The exact translation cocycle $\mathbf{c}(g,h)$ is stored in `composition_translation_table`. It is required whenever a derived consumer acts on absolute lifted placements.

Public API

```
normalize_periodic_automorphism(view, automorphism, *, anchor_atom_index=None)
identity_periodic_automorphism(view, *, anchor_atom_index=None)
compose_periodic_automorphisms(
    view, outer, inner, *, anchor_atom_index=None
)
invert_periodic_automorphism(view, automorphism, *, anchor_atom_index=None)
periodic_automorphism_composition_translation(
    view, outer, inner, *, anchor_atom_index=None
)
```

```
@dataclass(frozen=True)
class PeriodicNetSymmetry:
    periodic_net_view_digest: str
    topology_graph_digest: str
    anchor_atom_index: int
    operations: tuple[ValidatedPeriodicAutomorphism, ...]
    multiplication_table: tuple[tuple[int, ...], ...]
    composition_translation_table: tuple[tuple[LatticeShift, ...], ...]
    inverse_operation_indices: tuple[int, ...]
    identity_operation_index: int
    vertex_orbits: tuple[tuple[int, ...], ...]
    edge_orbits: tuple[tuple[FrameworkEdgeKey, ...], ...]
    canonical_schema_version: str
    digest_algorithm: str
    digest: str
```

```
build_periodic_net_symmetry(
    view: PeriodicNetView,
    generators: Sequence[ValidatedPeriodicAutomorphism],
    *,
    anchor_atom_index: int | None = None,
    max_operations: int = 512,
) -> PeriodicNetSymmetry
```

Construction algorithm

1. Validate every generator against the exact `PeriodicNetView` digest, vertex domain, multiedge domain, and active periodic subspace.
2. Normalize generators and exact inverses to the common anchor gauge.
3. Insert the identity.
4. Enumerate Cayley closure breadth first using normalized generators and inverses.
5. Abort transactionally if `max_operations` is exceeded.
6. Sort operations deterministically.
7. construct and verify multiplication, inverse, and cocycle tables.
8. Construct vertex and edge orbit partitions.
9. Compute the canonical source-bound digest.

Persistent boundary

The persistent group result deliberately excludes:

- primitive-ring keys and ring action tables;
- ring orbits and ring stabilizers;
- automatic-discovery frame trials;
- barycentric coordinates;
- Euclidean embedding data; and
- face, tile, or cage actions.

These are derived results with additional source dependencies. In particular, ring action identity depends on the exact `PrimitiveRingCatalog.digest`, not only on the topology and view.

Serialization

Schema:

`mdstats.periodic_net_symmetry.v3`

Deserialization requires the exact `PeriodicNetView`. The loader verifies:

- schema and digest algorithm;
- canonical payload digest;
- view and topology source digests;
- every normalized operation;
- multiplication, inverse, and identity tables;
- every cocycle entry; and
- deterministic orbit partitions.

No older ring-bearing symmetry payload is silently upgraded.

Input constraints and failure policy

- Every operation must be a fully validated automorphism of the same view.
- Lattice matrices must be integer unimodular and preserve the active periodic subspace.
- Explicit multiedge target and orientation data are mandatory.
- Closure must remain finite within `max_operations`.
- Failure returns no partial group.

Complexity

For group order $|G|$, closure is proportional to the number of discovered operation-generator products. Full table construction costs $O(|G|^2)$ exact compositions. The core serialized payload scales with group and quotient graph size but no longer scales with the number of primitive rings.

Validation requirements

Focused tests must cover:

- identity-only, cyclic, and dihedral groups;
- noncommuting generators;
- common-translation normalization;
- exact inverse matrices and multiedge orientation;
- deterministic generator-order independence;
- cocycle identities and source validation;
- resource-bound transactional failure; and
- serialization round trips.

References

1. S. J. Chung, Th. Hahn, and W. E. Klee, *Nomenclature and Generation of Three-Periodic Nets: The Vector Method*, Acta Cryst. A **40**, 42–50 (1984), DOI: `10.1107/S0108767384000088`.

2. O. Delgado-Friedrichs and M. O'Keeffe, *Identification of and Symmetry Computation for Crystal Nets*, Acta Cryst. A **59**, 351–360 (2003), DOI: 10.1107/S0108767303012017.